

2차 발표 후

팀 개별 피드백

HomeTalk OnePass 프로젝트 · 팀원 대상 코드 리뷰 & 발표 피드백

3차 최종 발표 D-25 → 2025년 6월 10~11일

회원·인증

주차·홈화면

공지·일정

알림·관리비

커뮤니티

시설예약·민원

전체 총평

강사 종합 평가 · 2차 발표 기준

완성도

매우 높음

7주 과정 치고는
상업 수준에 근접한 완성도

기획력

탁월

설문 기반 수요조사 +
사업화 전략까지 수록

기술 깊이

견고함

SSE 실시간 알림 ·
Soft Delete · OAuth2 등

3차 발표 전 발표자료 공통 개선 포인트

- ① 슬라이드 수 축소 : 114장 → 50~60장으로 압축, 핵심 화면 중심으로 재편
- ② 순서 조정 : 수요조사(Slide 106~107)를 프로젝트 소개 직후(Slide 3~4)로 이동 — 설득력 강화
- ③ R&R 슬라이드 위치 : 목차 전 → 목차 후로 이동해 흐름 자연스럽게
- ④ 개인 파트별 '기술 하이라이트' 1장 추가 : 단순 기능 나열이 아닌 설계 판단 포인트 서술

회원·인증

회원 & 마이페이지 시스템 | auth 패키지 전담

한줄 총평

Spring Security + OAuth2 전체를 설계·구현한 팀의 보안 아키텍트.
인증 기반이 흔들리면 모든 모듈이 무너지는 가장 책임감 있는 포지션을 완수함.

담당 도메인

회원 & 마이페이지 시스템 | auth 패키지 전담

회원·인증 · 잘한 점

01

Spring Security 설계의 완성도

SecurityConfig · CustomOAuth2User · CustomLoginSuccessHandler · RememberMeConfig 등 Security 계층을 체계적으로 분리. 소셜(Naver/Kakao) + 코걸 계정 듀얼 인증 구조를 단일 SecurityFilterChain 안에 깔끔하게 통합.

02

서비스 책임 분리의 모범 사례

SignUpService · WithdrawalService · EmailVerificationService · UserApprovalStatusService 등 역할별로 Service 클래스를 분리해 단일 책임 원칙(SRP) 실천. 팀 전체 코드의 유지보수성 향상에 기여.

03

승인 인터셉터로 ux 흐름 제어

ApprovalStatusInterceptor로 미승인 사용자의 무분별한 접근 차단. 단순 401 에러가 아닌 '승인 대기 안내 페이지'로 리다이렉트 하는 사용자 친화적 설계 적용.

04

팀 전체 인증 흐름의 단일 진입점

모든 모듈이 auth 패키지에 의존하는 구조. 팀원들의 코드에서 User·Household 엔티티를 참조할 수 있었던 것은 이 기반을 안정적으로 제공했기 때문.

회원·인증 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ 발표 시 인증 흐름을 기능 나열로 설명함
→ '로그인 요청 → Security Filter → OAuth2 → DB 저장 → 세션' 흐름도 1장이 더 효과적
- ▶ 소셜 로그인 기술적 판단 포인트가 미노출
→ Naver/Kakao 각 API 차이, 상태값 처리 등 고민 흔적을 드러내면 깊이 전달
- ▶ 마이페이지 기능이 발표에서 상대적으로 약함
→ '내가 어떤 행동을 했는지 볼 수 있다'는 UX 관점 설명 추가

3차 발표까지 목표

- ✓ 인증 흐름도(Login Flow Diagram) 1장 추가
로컬 / 소셜 두 경로를 하나의 다이어그램으로 표현
- ✓ 보안 취약점 수정 우선 진행
(이메일 중복검사 활성화 · CSRF 범위 축소 · Household 고아 데이터)
- ✓ 마이페이지 화면 스크린샷 보강
담당 파트 기술 하이라이트 1장 추가

주차.홈화면

홈화면 & 주차 시스템 | parking 패키지 전담

한줄 총평

6개 담당 중 도메인 복잡도 1위. 차량 등록부터 티켓 발급·입출차·수동 운영까지 실제 현장 운영 로직을 코드로 완전히 구현한 팀의 도메인 전문가.

담당 도메인

홈화면 & 주차 시스템 | parking 패키지 전담

주차·홈화면 · 잘한 점

01

Interface 기반 서비스 설계

VehicleService(interface) → VehicleServiceImpl, TicketService → TicketServiceImpl 구조로 의존 역전 원칙(DIP) 적용. 팀 전체에서 가장 정교한 객체지향 설계 적용 사례.

02

10개 Controller로 명확한 관심사 분리

AdminParkingController · AdminTicketController · AdminVehicleController · StaffEntryController · TicketController · VehicleController 등 역할별 Controller 완전 분리. 가독성과 협업 효율성 최상.

03

스케줄러를 활용한 자동화 로직

TicketScheduler로 만료 티켓 자동 처리 구현. 단순 CRUD를 넘어 시간 기반 비즈니스 로직까지 서버에서 담당. 실제 주차 운영 시스템의 핵심 요소 완성.

04

현장 경비원 시나리오까지 설계

StaffEntryController · StaffExitController · StaffViewController로 현장 근무자 UX를 별도 설계. '관리자' 외에 제3의 역할을 시스템 설계에 반영한 유일한 사례.

주차·홈화면 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ 홈화면과 주차 두 파트의 연결 고리가 약함
→ '홈화면에서 주차 현황을 바로 보여준다'는 UX 연결을 강조하면 시너지 효과 전달
- ▶ 주차 시스템의 복잡성이 발표에서 과소평가됨
→ 입출차 분기 로직(방문차/입주민차/미등록차)을 플로우차트 1장으로 시각화
- ▶ Interface 설계의 이점을 발표에서 미언급
→ '왜 Interface를 썼는가'를 면접 대비 설명으로 1문장 준비

3차 발표까지 목표

- ✓ 입출차 분기 플로우차트 1장 추가
(입주민 / 방문객 / 미등록 차량 3가지 경로 시각화)
- ✓ 현장 운영 시나리오 스크린샷 보강
경비원 화면 vs 관리자 화면 비교 구성
- ✓ 코드 버그 수정 우선 진행
(VehicleServiceImpl 순서 오류 · FileStorageService 롤백 처리)

공지·일정

대시보드 & 공지사항 & 일정 | dashboard · notice · schedule 패키지

한줄 총평

팀 전체에서 가장 창의적인 기능인 '공지-일정 자동 연계'를 구현.
사용자가 원하는 것을 예측해 자동으로 처리해주는 진짜 UX 개선을 만들어냄.

담당 도메인

대시보드 & 공지사항 & 일정 | dashboard · notice · schedule 패키지

공지·일정 · 잘한 점

01

공지-일정 자동 연계 기능 (팀 내 최고 창의력)

공지 등록 시 연관 일정을 자동 생성하는 로직 구현. '담당자가 공지를 올리면 입주민 캘린더에 자동 반영'은 실제 아파트 관리 앱에서도 흔치 않은 스마트 기능.

02

ReadLog로 중복 조회 방지 구현

공지 읽음 처리에 ReadLog 엔티티를 별도 설계해 '이미 읽은 공지' 이력 관리. 단순 boolean 플래그가 아닌 이력 관리 방식 채택.

03

반복 일정 자동 생성 (매주/매월)

ScheduleService에서 RepeatType 열거형으로 반복 패턴을 모델링. 구글 캘린더 수준의 반복 일정 개념을 Spring Boot로 직접 구현.

04

관리자/입주민 역할 분기 대시보드

DashboardController에서 Admin/Resident 권한에 따라 다른 데이터를 동적 렌더링. 단일 URL로 두 종류의 대시보드를 서빙하는 설계.

공지·일정 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ 대시보드의 관리자/입주민 분기 복잡성이 발표에서 잘 드러나지 않음
→ 같은 URL에서 두 화면이 다르게 보이는 데모가 큰 임팩트 포인트
- ▶ 공지-일정 연계 기능을 발표에서 단순 나열로 소개
→ '공지 등록 → 자동 일정 생성 → 캘린더 반영' 흐름을 화살표 흐름도로 보여주면 wow 포인트
- ▶ 반복 일정의 예지 케이스 처리 언급 없음
→ '매월 31일 반복 시 30일 월 처리' 등 고민 흔적을 면접 대비로 준비

3차 발표까지 목표

- ✓ 공지→일정 자동 연계 흐름도 1장 추가
(시퀀스 다이어그램 또는 화살표 플로우)
- ✓ 관리자 vs 입주민 대시보드 화면 비교 슬라이드
같은 URL, 다른 화면 구성 시각적으로 대비
- ✓ 코드 버그 수정 우선 진행
(limit 파라미터 수정 · 예외 로깅 추가 · readOnly 선언)

알림·관리비

앱 알림 & 관리비 시스템 | notification · billing 패키지

한줄 총평

SSE 기반 실시간 알림이라는 팀 전체의 기술적 하이라이트를 구현.
관리비 시스템의 엑셀 파이프라인까지 두 개의 복잡한 도메인을 동시에 완성함.

담당 도메인

앱 알림 & 관리비 시스템 | notification · billing 패키지

알림·관리비 · 잘한 점

01

SSE 기반 실시간 알림 — 팀 전체 기술 하이라이트

NotificationSseController · SseEmitterManager로 서버→클라이언트 단방향 실시간 푸시 구현. 일반적인 polling이 아닌 SSE를 선택한 것 자체가 기술 판단력을 보여주는 포인트.

02

이벤트 기반 아키텍처 설계

NotificationEvent · NotificationEventListener · NotificationPublisher로 pub/sub 패턴 구현. 각 모듈이 직접 알림을 보내지 않고 이벤트를 발행하는 방식 — 모듈 간 결합도를 낮춘 고급 설계.

03

AsyncConfig로 비동기 처리 분리

알림 발송을 별도 스레드풀에서 처리해 메인 비즈니스 로직의 응답 속도에 영향 없도록 설계. @Async + ThreadPoolTaskExecutor 패턴을 실무 수준으로 적용.

04

엑셀 업로드 → 파싱 → 검증 파이프라인

BillingUploadService에서 엑셀 1회 업로드로 전 세대 관리비 일괄 등록 구현. 금액 누락 검증 로직까지 포함. 코드 주석이 팀 전체에서 가장 상세 — 협업 기여도 최상.

알림·관리비 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ SSE라는 기술적 차별점이 발표에서 충분히 부각되지 않음
→ 'WebSocket 대신 SSE를 선택한 이유'를 30초 설명으로 준비 — 면접 단골 질문
- ▶ 알림 시스템의 전체 흐름도 미포함
→ '모듈 이벤트 발행 → Publisher → SSE → 클라이언트 수신' 흐름을 1장 추가
- ▶ 관리비 엑셀 업로드 화면이 기능 위주 소개
→ 파싱 + 검증 단계를 단계별로 시각화하면 기술력 전달

3차 발표까지 목표

- ✓ 알림 시스템 아키텍처 다이어그램 1장 추가
(모듈별 이벤트 발행 → Listener → SSE 흐름)
- ✓ 관리비 엑셀 업로드 Before/After 비교
(수기 입력 vs 엑셀 업로드 시간 절감 수치 포함)
- ✓ 코드 버그 수정 우선 진행
(BillingLog DELETE 타입 수정 · @Scheduled 분리)

커뮤니티

커뮤니티 게시판 | community 패키지 전담

한줄 총평

게시판→카테고리→게시글→댓글→태그의 5계층 구조를 완전히 설계·구현.
Soft Delete · 임시저장 · 마켓 게시판까지 완성도 높은 SNS급 커뮤니티를 만들.

담당 도메인

커뮤니티 게시판 | community 패키지 전담

커뮤니티 · 잘한 점

01

5계층 도메인 모델 설계

Board → Category → Post → Comment → Tag·PostTag의 완전한 계층형 구조 설계. 각 레이어에 Repository·Service·Controller 완비. 팀 전체에서 가장 복잡한 도메인 모델.

02

Soft Delete 기반 삭제 관리

BaseSoftDeleteEntity를 상속받아 물리 삭제 없이 논리 삭제 구현. 삭제된 게시글도 관리자가 조회·복구 가능. 실무 프로덕션 코드에서 필수로 사용되는 패턴을 팀 내 유일하게 적용.

03

임시저장 기능 구현

게시글 작성 중 임시저장 후 이어쓰기 기능. PostStatus enum으로 TEMP·PUBLISHED·HIDDEN 등 상태 관리. 사용자가 글을 잃지 않는 UX — 실제 서비스 수준의 세심한 기능.

04

마켓 게시판 확장 설계

TradeType · TradeStatus · MarketStatus enum으로 나눔/거래 카테고리를 별도 모델링. 일반 커뮤니티와 마켓을 같은 게시판 인프라 위에서 확장 설계 — 설계 유연성의 증거.

커뮤니티 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ 기능이 많아 핵심 차별점이 묻히는 경향
→ '왜 계층형 구조가 필요했는가'를 규모 기반으로 1문장 설명 추가
- ▶ Soft Delete의 이점이 발표에서 미언급
→ '삭제된 게시글도 관리자가 복구할 수 있다'는 UX 설명 추가
- ▶ 태그 기능의 활용도가 화면에서 잘 보이지 않음
→ 태그 검색 → 결과 필터링 UX를 데모 화면으로 강조

3차 발표까지 목표

- ✓ 좋아요 기능 + 인기글 정렬 보완 작업
- ✓ 커뮤니티 전체 도메인 관계도 1장 추가
(5계층 구조를 ERD 스타일로 발표에 포함)
- ✓ 코드 버그 수정 우선 진행
(category null NPE · 태그 N+1 · Dead Code 제거)

시설예약·민원

시설예약 & 민원 게시판 | facility · reservation · complaint · inquiry 패키지

한줄 총평

4개 패키지를 담당하면서 예약 시간 중복 방지부터 민원 상태 추적까지 사용자가 가장 자주 마주치는 화면들을 꼼꼼하게 완성함.

담당 도메인

시설예약 & 민원 게시판 | facility · reservation · complaint · inquiry 패키지

시설예약·민원 · 잘한 점

01

예약 시간 중복 방지 로직의 정교함

ReservationService에서 현재 시간 이전 예약 차단 · 1주일 이내 제한 · 같은 시설 중복 예약 방지 · 시간 겹침 체크를 4단계로 구현. 실제 시설 예약 서비스에서 반드시 필요한 모든 케이스를 커버.

02

민원과 1:1 문의를 별도 시스템으로 분리

complaint(민원) · inquiry(1:1 문의) 두 패키지를 독립 분리. 민원은 공개/상태 추적, 문의는 비공개 1:1 응답이라는 다른 요구사항을 구조적으로 분리. 설계 판단력이 돋보이는 포인트.

03

민원 상태 추적 시스템 (접수→완료)

ComplaintStatus enum으로 접수/확인/처리중/완료 4단계 상태 관리. 상태 변경 히스토리 추적으로 민원인이 진행 상황을 실시간으로 확인 가능.

04

ReservationNotificationScheduler로 예약 리마인더

예약 시간 전 자동 알림 발송 스케줄러 구현. 알림 모듈과 연동해 모듈 간 협업의 완성도를 보여주는 사례.

시설예약·민원 · 보완 포인트 & 3차 발표 목표

보완 포인트

- ▶ 민원 vs 1:1 문의의 차이가 발표에서 혼재
→ '민원은 공개, 문의는 비공개 1:1'이라는 차이를 슬라이드 한 줄로 명확히 정의
- ▶ 예약 중복 방지 로직의 기술적 깊이가 발표에서 미언급
→ 4단계 검증 로직을 '예약 불가 케이스' 표로 정리하면 면접 대비 강점
- ▶ 시설 예약 캘린더가 발표에서 작게 소개됨
→ 예약 가능 / 불가 시간대가 한눈에 보이는 캘린더 화면을 전면에 배치

3차 발표까지 목표

- ✓ 민원 vs 문의 차이 명확화 슬라이드 추가
(공개/비공개 + 상태 추적 유무 표로 정리)
- ✓ 예약 불가 케이스 4가지를 표로 시각화
(면접에서 '예외 처리 경험' 질문 대비)
- ✓ 코드 버그 수정 우선 진행
(첨부파일 디스크 잔류 · finishUsage 시간 오류 · findAll 메모리 필터링)

3차 최종 발표까지 — 공통 목표

1

발표자료 압축 & 재구성

114장 → 55장 / 수요조사를 앞으로 / 개인 기술 하이라이트 1장씩 추가

2

AWS / GCP 배포 완료

EC2 Run + S3 파일 저장 + HTTPS 인증서 적용 → 실제 접속 가능한 URL

3

포트폴리오 관점의 README

GitHub README에 아키텍처 다이어그램 · 기술 스택 · 담당 파트 · 배포 URL 기재

지난 7주의 성과는 대단함. 코드 품질을 높이고 배포까지 완성하면 진짜 서비스임

회원·인증 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

CRITICAL

설정파일 2개 — .gitignore 미등록 · 시크릿 전체 GitHub 노출

application.yaml · application-dev.yaml · .gitignore에 두 파일 모두 없음 → 현재 GitHub 노출 상태

직접 확인 결과: Gmail 앱 비밀번호 · DB 비밀번호(homeone402) · MySQL IP(210.178.108.186)
Kakao/Naver/Google OAuth2 시크릿 3개 전부 하드코딩. OAuth2 시크릿 재발급 + DB 비밀번호 변경 필요.

HIGH

SecurityConfig — CSRF 비활성화 범위가 너무 넓음

config/SecurityConfig.java · csrf ignoringRequestMatchers("/admin/**", "/community/**")

/admin/**, /community/** 전체 CSRF 비활성화. Thymeleaf Form 엔드포인트까지 포함되어 위조 요청 위험.
.csrf(csrf -> csrf.ignoringRequestMatchers("/api/**")) — REST API 경로만 정확히 지정해야 함.

HIGH

SignUpService — 이메일 중복 검사 주석 처리

auth/service/SignUpService.java · // validateEmailAvailable(dto.getEmail());

회원가입 시 이메일 중복 체크 완전 비활성화. 동일 이메일로 여러 계정 생성 가능한 상태. 즉시 주석 해제 필요.

회원·인증 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

WARNING

User.withdraw() — REJECTED와 WITHDRAWN 상태 구분 안 됨

auth/entity/User.java · withdraw() 메서드

탈퇴 시 status를 UserStatus.REJECTED로 설정 → 관리자 승인 거부와 자발적 탈퇴를 코드로 구분 불가.
UserStatus에 WITHDRAWN 추가 후 withdraw()에서 WITHDRAWN으로 변경해야 함.

WARNING

updateRejectedSignUp — 기존 Household 고아 데이터 잔류

auth/service/SignUpService.java · updateRejectedSignUp() 메서드

재가입 시 새 Household를 생성해 할당하지만 기존 Household는 삭제하지 않음.
반려-재신청 반복 시 Household 고아 레코드가 DB에 무한 누적됨.

주차·홈화면 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

HIGH

VehicleServiceImpl.register() — 트랜잭션 원자성 깨짐

`parking/service/VehicleServiceImpl.java` · `register()` 메서드

`vehicleRepository.save()` 후 `fileStorageService.saveDocuments()` 실행. `FileStorageService`에 `@Transactional` 없어 파일 디스크 저장 후 DB 롤백 시 파일은 남고 DB 레코드는 없는 불일치 상태 발생.

HIGH

VehicleServiceImpl.register() — 빈 파일 체크 순서 오류

`parking/service/VehicleServiceImpl.java` · `documentPaths.isEmpty()` 체크 위치

vehicle을 DB에 먼저 `save()`한 뒤 `documentPaths.isEmpty()` 검사로 예외를 던짐.
파일 없는 경우 예외 발생하지만 vehicle 레코드는 이미 DB에 저장된 상태.

PERF

StaffEntryService.search() — 전체 로드 후 메모리 필터링

`parking/service/StaffEntryService.java` · `search()` 메서드

PARKED 차량 전체를 메모리로 올린 뒤 `stream().filter()`로 번호 뒤 4자리 매칭.
DB 쿼리에서 직접 처리하는 `@Query`로 교체해야 함 — 차량 수 증가 시 심각한 성능 저하 발생.

공지·일정 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

HIGH

NoticeService.getRecentNotices(int limit) — 파라미터 무시

notice/service/NoticeService.java · getRecentNotices(int limit) 메서드

메서드가 limit 파라미터를 받지만 내부에서 findTop5ByStatusOrderByCreatedAtDesc() 하드코딩. limit이 무엇이든 항상 5개만 반환됨. PageRequest.of(0, limit) 사용 또는 파라미터 제거 필요.

WARNING

NoticeService.getNoticeDetail() — 조회수 증가 예외 완전 묵살

notice/service/NoticeService.java · getNoticeDetail() 내 catch(Exception e) {}

읽음 처리 및 조회수 증가 로직 전체를 try-catch로 묶고 예외 발생 시 아무것도 하지 않음. 조회수 버그 발생해도 로그조차 없어 원인 파악 불가. 최소 log.warn() 추가 필수.

TODO

NoticeService — 클래스 레벨 @Transactional readOnly 미선언

notice/service/NoticeService.java · 클래스 레벨 @Transactional (readOnly 없음)

클래스에 @Transactional만 선언 → 모든 읽기 메서드도 쓰기 트랜잭션으로 실행됨. 읽기 전용 메서드에는 명시적 @Transactional(readOnly=true) 선언 권장.

알림·관리비 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

HIGH

BillingService.deleteByBillingMonth() — 잘못된 로그 데이터 저장

billing/service/BillingService.java · deleteByBillingMonth() 내 BillingLog.builder()

관리비 삭제 후 감사 로그를 billing(null) + actionType(UPLOAD)으로 저장.
삭제 행위인데 UPLOAD 타입이고 어떤 Billing이 삭제됐는지 추적 불가능한 null 참조.

WARNING

NotificationService — @Scheduled + @Transactional 동일 메서드

notification/service/NotificationService.java · deleteExpiredNotifications()

@Scheduled 메서드에 @Transactional을 함께 붙이면 Spring AOP 프록시 미적용 위험.
별도 @Transactional 메서드를 만들고 스케줄러에서 호출하는 방식으로 분리 필요.

TODO

SseEmitterManager — Last-Event-ID 미구현 (연결 끊김 시 알림 누락)

notification/service/SseEmitterManager.java · connect() 내 TODO 주석

connect()에 Last-Event-ID 기반 누락 알림 재전송이 TODO로 남아있음.
브라우저 재연결 시 끊긴 동안 발행된 알림이 유실됨.

커뮤니티 · 코드에서 발견된 개선 필요 사항

소스코드 직접 분석 결과 · 3차 발표 전 반드시 수정 권장

HIGH

PostService.postSave() — category null 상태에서 .getCode() NPE

community/service/PostService.java · postSave() 신규 생성 분기

dto.getCategoryId()가 null이면 category 변수가 null인 채로 유지됨.
이후 신규 생성 분기에서 category.getCode()를 null 체크 없이 호출 → NullPointerException 발생.

WARNING

PostService — IllegalArgumentException 남용

community/service/PostService.java · 여러 orElseThrow 구문

비즈니스 예외에 IllegalArgumentException 사용 → GlobalExceptionHandler 미처리 시 500으로 떨어짐.
커스텀 예외(UserNotFoundException, PostNotFoundException)로 교체해야 함.

PERF

PostService 태그 처리 — N+1 Query 위험

community/service/PostService.java · postSave() 태그 루프

태그마다 findByName() SELECT → 없으면 save() INSERT 루프 → 태그 개수만큼 DB 왕복.
IN 쿼리로 한 번에 기존 태그 조회 후, 없는 것만 batch INSERT 하는 방식으로 개선 필요.

전체 공통 · 모든 팀원 해당 개선 필요 사항

3차 발표 전 전원 확인 필수 · 소스코드 + 설정 파일 분석 결과

CRITICAL	긴급 — 설정파일 GitHub 노출 · OAuth2 시크릿 재발급 · DB 비밀번호 변경 application-dev.yaml이 .gitignore에 없음 → DB 비밀번호·MySQL IP·Kakao/Naver/Google 시크릿 GitHub 공개 상태. 지금 : ① OAuth2 시크릿 3개 재발급 ② DB 비밀번호 변경 ③ .gitignore 추가 후 Git 이력에서 파일 제거(BFG)	application.yaml · application-dev.yaml · .gitignore 미등록 확인
CRITICAL	application.yaml — ddl-auto: update 운영 배포 전 반드시 변경 운영 서버에서 ddl-auto: update는 컬럼 삭제 등 스키마 변경 시 데이터 손실 위험. application-prod.yaml → ddl-auto: validate 로 분리 필수.	resources/application.yaml
WARNING	context-path: /hometop — 프로젝트명 hometalk와 불일치 배포 전 /hometalk 로 통일 필요. 현재 상태로 배포 시 모든 URL 경로가 /hometop으로 노출됨.	resources/application.yaml · server.servlet.context-path
WARNING	Git — 충돌 흔적 파일 'how e4557f4' 소스 루트에 존재 Git merge 충돌 흔적 파일이 소스 루트에 남아있음. 즉시 삭제 후 .gitignore에 *.log, /build 추가 필수.	프로젝트 source root · 'how e4557f4' 파일
TODO	DTO 네이밍 혼용 — RqDTO / RsDTO / RequestDTO / Dto / DTO 혼재 팀 컨벤션 통일 권장 — XxxRequestDto / XxxResponseDto 형식으로 일괄 정리.	전 모듈 DTO 클래스
TODO	단위 테스트 부재 — 컨텍스트 로드 테스트 1개만 존재 3차 발표 전 핵심 Service 단위 테스트 최소 3개 이상 작성 권장. 면접에서 TDD 경험 질문에 대비.	test/java/ · OnepassApplicationTests.java 외 없음
TODO	Thymeleaf 오류 페이지 미처리 — Whitelabel Error Page 그대로 노출 GlobalExceptionHandler가 JSON 전용으로 구성됨 → HTML 요청 오류 시 Spring 기본 오류 페이지 노출. templates/error/ 폴더에 커스텀 페이지 추가 권장.	templates/error/ — 5xx.html · 4xx.html 없음